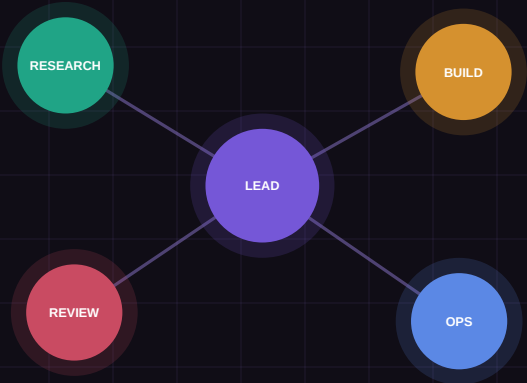


THE GROK BOT AGENT BLUEPRINT

A first-principles guide to memory, messages, priority, meetings, screens, permissions, and model-independent orchestration.

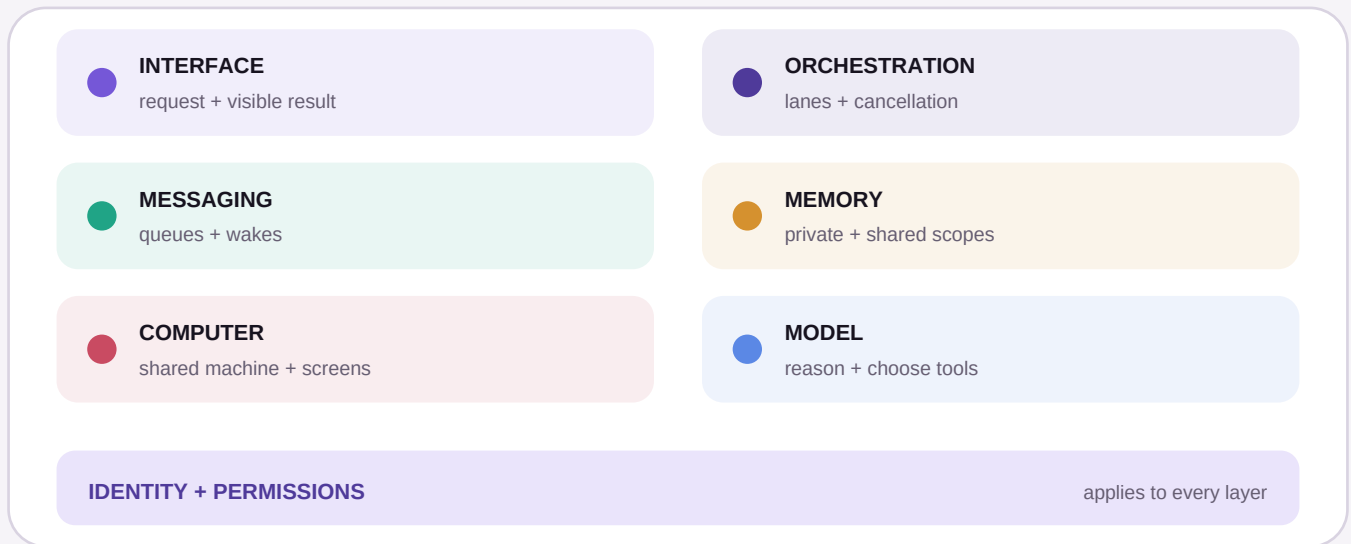
IDENTITY	MEMORY	MESSAGES
MEETINGS	SCREENS	PERMISSIONS



READ THIS FIRST

The spectacle is not one big brain.

It is a system of identities, queues, notebooks, policies, and computers. The language model works inside that structure. This guide shows what each layer does, why the boundaries matter, and how to add the same primitives to an agent you already use.



TRUTH BOUNDARY

The source reviewed for this guide is an unofficial Grok Bot 0.18 reconstruction, not Cursor's authenticated original repository and not xAI model source. Provider routing for Codex, Claude Code, OpenRouter, and local Docker is identified by the repository as reconstruction-author additions. This pack contains original explanation and diagrams, not recovered source or binaries.

EVIDENCE LABELS

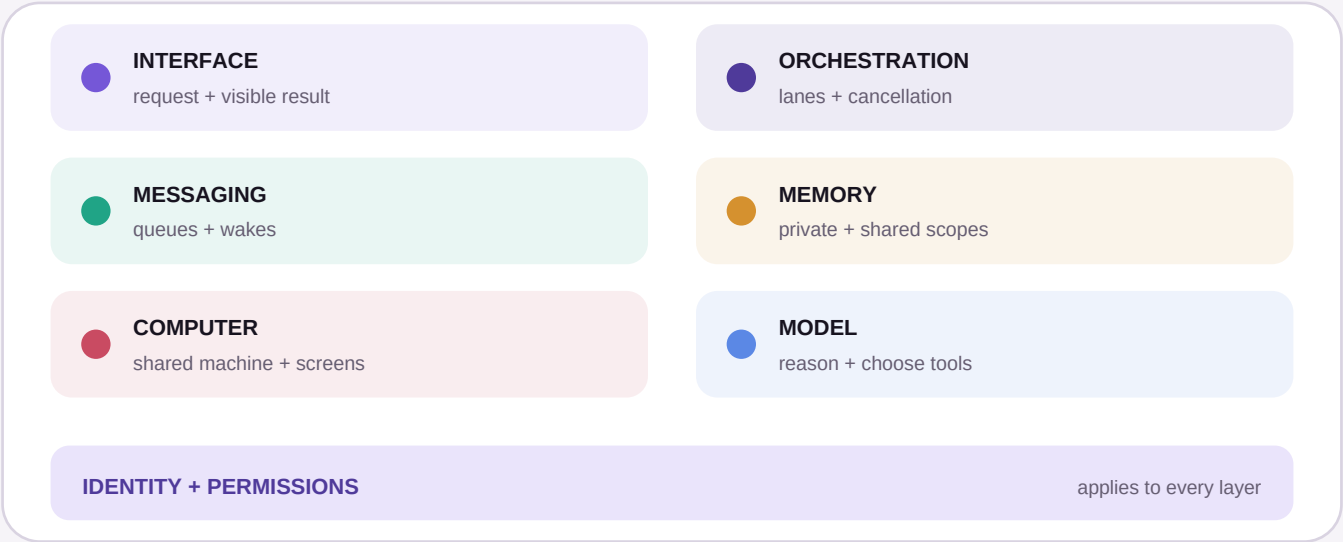
Verified behavior means the pinned TypeScript directly expresses it. Repository statement means the reconstruction documents it. Builder interpretation means this guide derives a useful design rule from the implementation. Unknown means the reviewed evidence does not establish the claim.

RESEARCH LOCK 25 AUGUST 2026 | COMMIT a9f633e09d49a85829b8236331b9e21f7e612634

CONTENTS

Part 1: The system in one page	4
Part 2: An agent is a bundle, not a model	6
Part 3: The mailroom	8
Part 4: Priority and interruption	10
Part 5: Bounded meetings	11
Part 6: One computer, separate screens	13
Part 7: Permissions and trust	15
Part 8: Tools, workflows, plugins, and skills	17
Part 9: The model layer	18
Part 10: A buildable architecture	19
Part 11: Adapting an existing agent	22
Part 12: Failure modes	23
Part 13: What the video simplified	24
Part 14: Glossary	25
Part 15: Final blueprint	25
Part 16: Repository source map	26

Part 1: The system in one page



The six layers

The easiest way to understand a multi-agent product is to stop thinking of it as one large AI brain. Think of it as six layers that cooperate.

Layer	First-principles job	Plain-language picture
Interface	Accepts requests and shows activity, chats, tools, and results.	The office lobby.
Orchestration	Chooses which agent runs, when it runs, and what can interrupt it.	The operations manager.
Messaging	Moves small packets of context between agents without merging their minds.	The mailroom.
Memory	Stores durable facts separately from the current chat window.	Personal and shared notebooks.
Computer	Gives agents tools, files, terminals, browsers, and visible desktop surfaces.	The office floor and desks.
Model	Produces reasoning, language, and tool choices for a turn.	The worker at the desk.

Identity and permissions cut across all six layers. Identity tells the system who an agent is. Permissions decide what that agent may touch.

The core loop

Every useful multi-agent system repeats the same loop:

- 1 A request arrives.
- 2 The system chooses an agent and a run lane.
- 3 The agent receives its profile, relevant memory, recent transcript, and available tools.
- 4 The model decides whether to work, message a teammate, call a tool, or ask for permission.
- 5 New information is recorded in a transcript, queue, memory store, or group room.
- 6 Another agent may wake on a later turn.
- 7 A lead agent or the interface assembles the visible result.

The intelligence is not only in the model. Much of the product experience comes from the rules around the model.

One end-to-end example

Imagine the user asks: "Research the best checkout pattern, build it, and review the final page."

- 1 The lead agent turns one request into three roles: researcher, builder, and reviewer.
- 2 Each role has its own profile, transcript, memory view, and desktop window.
- 3 The researcher begins browsing while the builder prepares the project.
- 4 The researcher sends a short asynchronous message containing the three facts that matter. The send returns immediately.
- 5 The builder receives the message on a later hidden wake and incorporates it without requiring the researcher to stop.
- 6 The builder finishes and sends a message to the reviewer.
- 7 The reviewer opens the running page on a separate screen, finds a checkout failure, and sends a priority message.
- 8 The priority message can interrupt the builder's non-user work. It cannot preempt an active user conversation.
- 9 If judgment is required, the agents enter a bounded group room. Agents speak in rounds, can pass, and stop at hard limits.
- 10 The lead agent returns one answer to the user with the conclusion, evidence, and remaining risks.

That is the full machine. Everything else is a refinement of one of those steps.

Part 2: An agent is a bundle, not a model

What makes two agents different

Two agents can use the same underlying model and still behave like different teammates. The difference comes from the bundle around each model call.

In the reviewed repository, each agent has its own directory and database. The agent profile includes a name, description, title, avatar shape, and avatar color. A separate database holds the agent's transcript and operational state. Memory is stored and recalled through agent-specific paths and prompts.

At a minimum, an agent record needs:

```
Agent
  id
  name
  role or description
  transcript store
  memory store
  enabled tools and workflows
  run state
  current desktop window
  permissions and policy context
```

The model is replaceable. The agent record is what preserves continuity.

Private chat history

A common mistake is to copy every message into one shared conversation and label the speakers differently. That creates one large context soup. It also makes privacy, debugging, and interruption much harder.

The reviewed design keeps per-agent session folders and per-agent databases. The system can read another agent's transcript through explicit coordination functions, but the histories are not automatically fused into one prompt.

This gives four benefits:

- An agent can keep working without carrying every other agent's tokens.
- A specialist can retain its own history and style.
- Messages between agents become deliberate handoffs rather than accidental context leakage.
- A failure can be traced to one agent's transcript and run lane.

Profile, transcript, and memory are different things

These three are easy to confuse:

- Profile: stable identity and role, such as "You are the QA reviewer."
- Transcript: the chronological record of what happened in conversations and tool turns.
- Memory: selected durable facts that should survive beyond the current chat.

Do not use one storage object for all three. They have different update rules, retention needs, and trust boundaries.

Memory has tiers and scopes

The reviewed reconstruction uses multiple memory tiers:

- **profile**: enduring facts that should stay in mind.
- **log**: substantive history such as projects, decisions, and commitments.
- **note**: lower-priority details that remain on disk but do not deserve constant prompt space.

It also supports multiple scopes:

- **agent**: private to one assistant.
- **user**: shared user memory, stored in per-assistant shards to preserve a single writer for each file.
- **project**: memory associated with a project, again separated into per-assistant shards.

This is more nuanced than "every agent has private notes." Agents do have private agent memory, but the system also has explicit shared memory scopes.

Memory is a budgeted view, not a full dump

The implementation limits how much memory enters a prompt. It ranks and budgets recent facts, keeps durable profile facts visible, and leaves older material on disk for explicit reading or search.

This creates a useful design rule:

Store broadly, retrieve narrowly, and show the model only what the current turn can justify.

Dumping an entire memory database into every prompt is expensive and often makes answers worse.

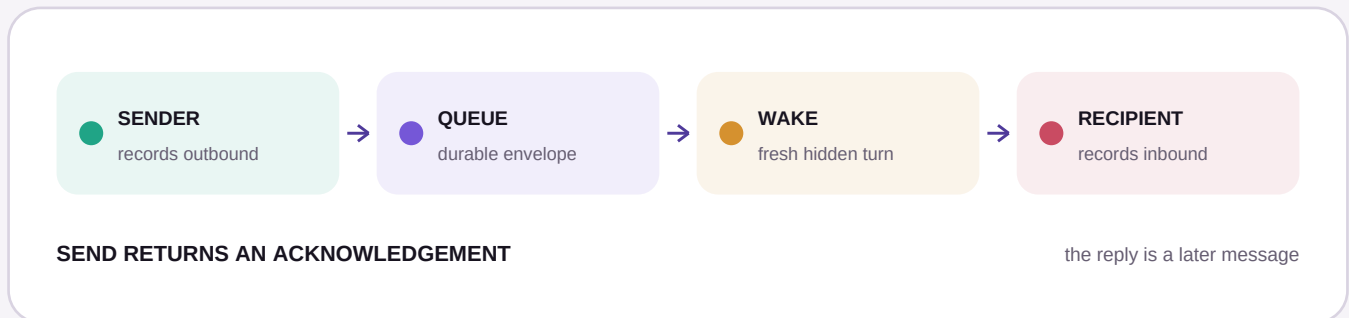
Builder pattern: a clean agent context

Before each turn, construct a context envelope:

```
context = {
  identity: agent.profile,
  recent_chat: transcript.tail(agent.id),
  durable_memory: memory.recall(agent.id, request),
  shared_project_memory: memory.recall(project.id, request),
  teammate_directory: roster.visible_to(agent.id),
  tools: policy.allowed_tools(agent.id),
  request: incoming_turn
}
```

The exact storage engine does not matter. The separation does.

Part 3: The mailroom



Why asynchronous messages matter

If Agent A calls Agent B and waits for a full answer inside the same turn, the system becomes a chain of blocking phone calls. One slow agent stalls everyone behind it.

The reviewed [SendToAgent](#) behavior is fire-and-forget:

- 1 The sender chooses a target agent ID or a group ID.
- 2 The system records the outbound message.
- 3 The system queues an inbound message for the recipient.
- 4 The send returns an acknowledgement immediately.
- 5 The recipient wakes on a later hidden turn.
- 6 A reply, if any, arrives later as a new message.

The sender is explicitly told not to poll or wait for a reply in the same turn.

A message is not shared consciousness

A useful handoff contains only what the next agent needs:

```

Goal: what outcome is needed
State: what has already happened
Evidence: the facts or files that matter
Constraint: what must not change
Next action: the precise ask
  
```

This format keeps the message small and auditable. It also prevents an agent from forwarding the user's private wording when a short professional summary is enough.

The wake mechanism

The recipient does not continuously watch an inbox. The host queues a message and later runs the recipient with a hidden prompt that identifies the sender, marks the event as an agent message, and explains whether it is normal or priority traffic.

That hidden wake is an important product primitive. It lets work continue after the original user turn has moved on. It also means the system needs durable pending-wake state, retry behavior, and protection from ping-pong acknowledgements.

Images and groups

The reviewed repository supports images in one-to-one agent messages. Group messages are text-only in the inspected version. Priority is also one-to-one only and is ignored for group posts.

These details matter when designing the user experience. A generic "message" abstraction often hides capability differences that later become bugs.

Original queue pseudocode

```
function send_to_agent(sender, target, message, priority=false):
    assert target != sender
    record_outbound(sender, target, message)
    envelope = {sender, message, priority, created_at: now()}

    if priority:
        inbox[target].push_front(envelope)
        maybe_interrupt_non_user_run(target)
    else:
        inbox[target].push_back(envelope)

    schedule_wake(target)
    return delivery_acknowledgement()
```

The reply is not the return value. That one decision changes the whole architecture.

Reliability requirements

A serious mailroom needs:

- Stable agent IDs rather than names as addresses.
 - Message IDs for deduplication.
 - A durable queue or recoverable pending-wake record.
 - A visible transcript entry for both sender and recipient.
 - Bounded message size.
 - Delivery acknowledgement that does not pretend to be a reply.
 - Retry rules that do not create duplicate work.
 - A rule against automatic acknowledgement loops.
-

Part 4: Priority and interruption

Priority is a scheduler decision

Priority is not bold text. It changes what the run scheduler may stop.

The reviewed behavior places priority messages at the front of the inbound queue and attempts to interrupt the recipient's active non-user work. The code first checks the active lane. If the recipient is in the `user` lane, it does not interrupt.

This creates a clear order of authority:

- 1 Active user conversation
- 2 Priority one-to-one agent message
- 3 Normal peer message
- 4 Automations and other background work

The exact scheduler has more detail, but this ordering explains the visible behavior.

Why the user lane stays protected

Without a protected user lane, an agent could vanish in the middle of answering the person because another agent marked a message urgent. That would make the product feel broken and would let internal automation outrank explicit human intent.

The safe rule is:

Internal work may interrupt internal work. Internal work must not silently outrank the person currently speaking.

Supersede, do not merely reorder

A priority message can represent a true direction change, such as "Stop polishing the animation. Checkout is broken."

If the old run is only paused and later resumes without knowing it was superseded, it may undo the urgent fix. The reviewed code marks the interruption reason, redrives remaining messages when appropriate, and distinguishes user work from background or group-member work.

Your implementation should record:

- Which run was interrupted.
- Why it was interrupted.
- Whether partial output is safe to reuse.
- Whether the old job should resume, restart, or be abandoned.

Priority abuse control

Do not let every agent mark every message urgent by default. Good controls include:

- Priority only for one-to-one messages.
- A small set of allowed reasons, such as stop, supersede, safety, or time-critical failure.
- Rate limits per sender.
- A visible audit event.
- No interruption of the user lane.

ARCHITECTURE FIELD GUIDE

Part 5: Bounded meetings

A meeting is a controlled room, not a prompt pile

When agents need judgment rather than a simple handoff, a group room can expose multiple perspectives. The dangerous version sends one prompt to every agent and lets them talk forever. The reviewed design is bounded.

The pinned implementation sets these limits:

Limit	Value
Maximum group members	6
Maximum rounds	3
Maximum member messages in one orchestration run	10
Maximum messages from one member turn	2
Recent room messages supplied as history	24

The orchestrator rotates the starting speaker by round. Mentions can narrow the responders. An agent may answer with [\(pass\)](#), and pass messages are removed instead of cluttering the room.

The manager nuance

The video uses a manager at the round table as a helpful explanation. The reviewed group orchestrator does not hard-code a permanent manager role. It resolves responders, orders speakers, runs bounded rounds, and lets members pass.

A manager can still exist as an agent profile or as a prompt-level role. It is a design choice layered on top of the room, not a required primitive in the inspected orchestrator.

Why limits improve intelligence

Limits are not only cost controls. They force the system to decide whether it has anything new to add.

Stop the meeting when:

- A round produces no new messages.
- The message cap is reached.
- The room's request has been superseded.
- The current orchestration epoch is no longer valid.

A useful meeting prompt

Each member should receive:

- Its own identity and role.
- The room's purpose.
- The other participants.
- Only the recent room history it needs.
- A clear instruction to send one useful message or pass.
- A reminder not to reveal private one-to-one context.

Original meeting pseudocode

```
for round in 0..MAX_ROUNDS:
    responders = resolve_mentions_or_everyone(room_history)
    speakers = rotate_start(responders, round)
    messages_this_round = 0

    for agent in speakers:
        if total_messages >= MAX_MESSAGES:
            stop

        replies = run_member_turn(agent, recent_room_history)
        useful = remove_passes_and_empty_text(replies)

        for reply in useful.take(MAX_PER_MEMBER_TURN):
            append_to_room(agent, reply)
            total_messages += 1
            messages_this_round += 1

    if messages_this_round == 0:
        stop
```

Pick the smallest coordination primitive

Situation	Use	Why
One agent needs a fact or a clear handoff from one owner.	Normal direct message	Small context packet, no interruption, no meeting overhead.
New information invalidates active background work.	Priority direct message	Supersedes non-user work while protecting the active user conversation.
Several specialties must make one judgment call.	Bounded room	Shared evidence, controlled turns, pass behavior, and a hard stopping rule.
The work must happen later or on a schedule.	Automation wake	Durable trigger that does not masquerade as a peer conversation.

If a normal direct message is enough, do not create a meeting. If a queued message is enough, do not interrupt a run.

When to use a meeting

Use a room for:

- Go or no-go decisions.
- Conflicting evidence.
- Tradeoffs that need multiple specialties.
- Final review of a high-risk action.

Use a direct message for:

- A clear handoff.
- A status update.
- A missing fact.
- A request with one obvious owner.

ARCHITECTURE FIELD GUIDE

Part 6: One computer, separate screens

The accurate mental model

The video says each agent has its own computer. The reviewed implementation is more precise:

Agents share one sandbox computer, while each active agent can be assigned its own private desktop window or monitor.

The shared desktop layer maintains a map from agent IDs to window indexes. It persists assignments, creates a window-owner token, and returns a separate VNC URL and computer-use accessor for that window.

The code itself describes the failure case as every monitor on the shared computer being in use.

What is private and what is shared

Surface	Typical boundary in the reviewed design
Desktop view	Separate per-agent window or monitor
VNC connection	Separate URL or display token
Window ownership	Token tied to the assigned agent window
Underlying computer	Shared sandbox or box
Files and terminal environment	Shared machine state unless another layer isolates paths
Agent transcript and profile	Separate per-agent storage

This matters. A private screen does not automatically mean a private file system.

Why separate screens help

- The researcher can keep browser tabs open.
- The builder can keep a terminal and app server running.
- The reviewer can inspect the rendered product without stealing the builder's visible session.
- The interface can show what each agent is doing.

Capacity is real

Screens are a finite resource. The implementation assigns free window indexes and reports when no monitor is available. A production system needs explicit behavior for resource exhaustion:

- Queue the computer-use step.
- Fall back to non-visual tools.
- Reuse a released window.
- Tell the user the screen is unavailable.

Never pretend that unlimited parallel browser work exists when the desktop backend cannot support it.

Part 7: Permissions and trust

Two computers, two trust questions

The system distinguishes the agent's sandbox computer from the user's actual computer.

The agent's own sandbox is the default work surface. Access to the user's machine uses external tools and passes through a separate local-tool permission gate.

This distinction is one of the most important safety boundaries in the product.

The three standing modes

The inspected local-tool permission system uses:

- **always**: allow eligible local-computer actions.
- **ask**: create an approval request.
- **never**: block the actions.

The default is **ask**. An administrator can also impose a ceiling so a user cannot choose a more permissive setting than policy allows.

The four answers to an approval card

When the system asks, the user can resolve the request as:

- Allow once
- Deny
- Always allow
- Never allow

An allow-once approval is scoped to a specific agent, tool call, action, and target. Some approvals can cover a related resource path. Approvals are retired at scope completion or when a new turn begins. Pending asks expire after ten minutes in the reviewed implementation.

What the recording simplified

The recording suggests the system asks the first few times and then learns that an action is safe. The inspected local-tool permission code does not support that general claim.

Repeated approval does not silently train a permanent allow list. A standing **Always** decision changes the permission mode. **Allow once** remains scoped. Denials are remembered for the current direction so the agent cannot keep asking for the same rejected action.

The repository evidence reviewed for this guide also does not prove a general credit-card purchasing policy. The inspected local-tool actions cover commands, input, reading files, listing directories, and writing files. Treat autonomous purchasing as a separate high-risk capability that would need its own explicit policy.

A practical permission matrix

Action	Default policy	Why
Read public web page	Allow	Low local impact.
Read sandbox file	Allow within workspace	Agent-owned environment.
Run sandbox test	Allow within workspace	Reversible and contained.
Read user's private file	Ask	Crosses into personal machine data.
Write user's file	Ask with exact path	Changes user-owned state.
Publish, pay, send, or delete	Ask with exact consequence	External or destructive side effect.
Change standing permission	User-only decision	Alters the trust boundary itself.

Approval design rules

- Show the exact action and target.
 - Keep approval scoped and time-bounded.
 - Do not convert repetition into consent.
 - Remember denials long enough to prevent nagging.
 - Make **Never** a real block, not a prompt suggestion.
 - Let policy impose a ceiling.
 - Log who asked, what was approved, and what ran.
-

Part 8: Tools, workflows, plugins, and skills

Tools are capabilities

A model cannot browse, run commands, message agents, or edit files because it is intelligent. It can do those things only when the host exposes a tool and authorizes a call.

Tools should be described with:

- A stable name.
- A typed input schema.
- A clear side-effect description.
- A permission policy.
- A result format.
- An audit record.

The reviewed [SendToAgent](#) tool is a good example. Its schema requires a target ID and message, permits images, and has an optional one-to-one priority flag.

Workflows and skills

The reconstruction has a workflow library that can store Markdown instructions, helper files, managed skills, plugin skills, and automations. Workflows can be enabled for particular agents.

The inspected local import function discovers:

- `CLAUDE.md`
- `AGENTS.md`
- `.claude/CLAUDE.md`
- Markdown or MDC files under `.cursor/rules/`

It can also import Markdown directly or link to a live source path.

Important import correction

The recording says you can import skills from Codex or Claude Code. The reviewed discovery function does not prove wholesale import of every folder under `.codex/skills/` or `.claude/skills/`.

What the repository clearly supports is a workflow abstraction, imported Markdown, linked instruction files, managed skills, and plugin skills. A builder can create a separate adapter for Codex or Claude skill folders, but that adapter should parse each ecosystem deliberately rather than assuming all skill formats are identical.

A safe skill adapter

For each external skill:

- 1 Detect the manifest or instruction entry point.
- 2 Copy or link only the declared files.
- 3 Record the source ecosystem and original path.
- 4 Normalize name and description.
- 5 Preserve helper scripts but do not execute them during import.
- 6 Ask the user which agents may enable the skill.
- 7 Run imported scripts inside the agent sandbox with normal permissions.

ARCHITECTURE FIELD GUIDE

Part 9: The model layer

The model should not own orchestration state

The model can suggest who should act next, but durable state belongs in the host:

- Queues belong in a message service.
- Active runs belong in a scheduler.
- Agent profiles belong in storage.
- Permissions belong in a policy service.
- Room history belongs in the room store.
- Desktop assignments belong in the computer service.

If all state exists only inside a prompt, a restart destroys the organization.

Model independence

A well-designed host can route different turns to different model providers. The reconstruction includes a provider router for Cursor, Claude Code, Codex, and OpenRouter, but its own README identifies these routing features as additions by the reconstruction author.

Do not attribute that router to Cursor's original Grok Bot without separate evidence.

The useful first-principles lesson is still valid: make the orchestration contract provider-neutral.

```
ModelAdapter
  stream(messages, tools, settings)
  normalize_tool_call(call)
  report_usage()
  cancel(reason)
```

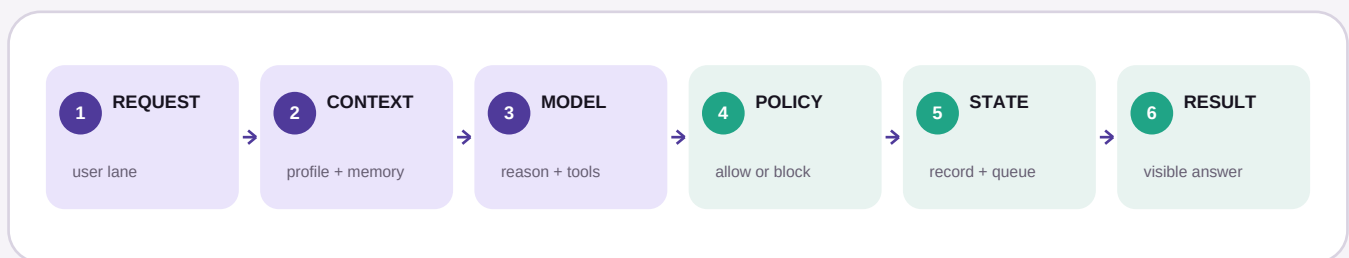
The host should be able to replace the adapter without rewriting memory, messaging, meetings, or permissions.

Unknowns

The reviewed materials do not establish xAI model weights, hidden reasoning implementation, or the exact model-effort policies of the upstream product. Those belong in the unknown column.

ARCHITECTURE FIELD GUIDE

Part 10: A buildable architecture



Services

A minimal implementation can use eight services:

- 1 Agent registry: IDs, names, roles, profiles, enabled workflows.
- 2 Transcript store: one append-only conversation record per agent and room.
- 3 Memory service: durable facts with agent, user, and project scopes.
- 4 Message service: outbox, inbox, acknowledgements, priority ordering.
- 5 Run scheduler: user, agent, automation, and background lanes.
- 6 Room orchestrator: bounded rounds, passes, mentions, and history windows.
- 7 Computer service: sandbox lifecycle, window assignment, terminal and browser access.
- 8 Permission service: standing modes, scoped approvals, denials, and audit events.

The interface and model adapters sit on top of these services.

Six system invariants

These rules should remain true even if the interface, model provider, or storage engine changes:

- 1 A user's active turn outranks all internal work.
- 2 A delivered message survives process restart and wakes its recipient no more than once.
- 3 Private transcripts and agent-scoped memory do not enter another agent's prompt by default.
- 4 The model may propose a state change, but only the host commits it.

- 5 Every external or destructive side effect has an authorization decision and audit record.
- 6 Any run, room, or automation can be cancelled without leaving hidden work that later resumes blindly.

Suggested data objects

```
AgentRecord
  id, profile, created_at, enabled_workflows

RunRecord
  id, agent_id, lane, status, started_at, interrupt_reason

MessageEnvelope
  id, sender_id, target_id, text, attachments, priority, created_at, status

RoomRecord
  id, member_ids, purpose, epoch, created_at

MemoryFact
  id, owner_scope, owner_id, tier, text, learned_at, source

DesktopAssignment
  agent_id, computer_id, window_index, owner_token, last_seen

ApprovalRequest
  id, agent_id, action, target, scope_id, expires_at, resolution
```

Run lanes

At minimum, use these lanes:

Lane	Trigger	Interruption rule
User	A person sends a message or steer.	Highest priority.
Agent	Another agent sends a message.	Priority peers may supersede non-user work.
Automation	A schedule or rule wakes the agent.	Runs after user and peer work.
Background	A tool, subtask, or connector completes.	Lowest urgency unless promoted.

Queueing every event into one FIFO list loses the user's priority and makes urgent coordination unreliable.

End-to-end request algorithm

```
on_user_request(request):
    lead = select_lead_agent(request)
    enqueue_run(lead, lane="user", payload=request)

run_agent(agent, payload):
    context = build_context(agent, payload)
    result = model.stream(context, allowed_tools(agent))

    for event in result.events:
        if event is tool_call:
            decision = permission_service.authorize(agent, event)
            execute_or_block(event, decision)

        if event is peer_message:
            message_service.deliver(event)

        if event is room_request:
            room_orchestrator.run_bounded(event.room)

    append_to_transcript(agent, event)

memory_service.extract_and_store(agent, result)
publish_visible_result(result)
```

Build order

Do not start with five animated agents. Build in this order:

- 1 One agent with a profile and durable transcript.
- 2 A second agent with an isolated transcript.
- 3 Fire-and-forget one-to-one messages.
- 4 Durable wake and retry behavior.
- 5 User and background run lanes.
- 6 Priority peer interruption with the user lane protected.
- 7 Bounded rooms with pass behavior.
- 8 Scoped memory retrieval.
- 9 Sandbox tools and separate desktop windows.
- 10 Permission cards and audit logs.
- 11 Provider adapters and advanced skills.

Each step should work before adding the next.

Part 11: Adapting an existing agent

Start with an audit

Before adding features to Hermes, OpenClaw, a custom app, or another agent runtime, find the primitives it already has.

Ask:

- Is there already a stable agent ID?
- Does each agent already have a transcript?
- Is there a background job queue?
- Can a running turn be cancelled?
- Are tool calls typed and auditable?
- Is there a sandbox?
- Is memory separate from the transcript?
- Can the UI show multiple agent states?

Do not rebuild a primitive that already exists.

Four integration levels

Level 1: Better specialists

Add profiles, isolated histories, and scoped memory. No agent-to-agent messaging yet.

Level 2: Asynchronous handoffs

Add an inbox, `SendToAgent`, hidden wakes, and visible message records.

Level 3: Coordinated teams

Add priority lanes, bounded group rooms, and a lead-agent result contract.

Level 4: Independent work surfaces

Add sandbox tools, private desktop windows, permission gates, and provider routing.

This staged path is safer than importing a complete multi-agent abstraction in one change.

Acceptance tests for an adaptation

- Agent A can message Agent B without waiting for B's answer.
- B's reply arrives on a later turn and wakes A once.
- Restarting the host does not lose a queued message.

- A normal peer message does not interrupt an active run.
- A priority peer message can interrupt non-user work.
- No peer message interrupts the user lane.
- A room stops after its configured bounds.
- An agent can pass without adding a visible message.
- Separate agents do not receive each other's private transcript by default.
- Memory retrieval stays within its declared scope.
- Separate desktop windows do not imply separate file systems.
- **Allow once** expires with its scope.
- **Never** blocks the action.
- A denied action is not immediately requested again.
- Every side effect has an audit event.

ARCHITECTURE FIELD GUIDE

Part 12: Failure modes

Shared context soup

Symptom: every agent receives every message.

Result: high token cost, personality drift, privacy leakage, and no clear ownership.

Fix: isolate transcripts and send deliberate handoffs.

Blocking DMs

Symptom: Agent A waits inside one tool call for Agent B's full result.

Result: serial work disguised as a team.

Fix: acknowledge delivery, wake later, and continue independently.

Priority everywhere

Symptom: every message interrupts current work.

Result: thrashing and unfinished tasks.

Fix: protect the user lane and require a small set of priority reasons.

Endless meetings

Symptom: agents keep agreeing, restating, and hallucinating.

Result: cost without decisions.

Fix: bound rounds, messages, history, and allow pass.

Fake computer isolation

Symptom: the UI shows separate screens, but every agent can overwrite the same files without coordination.

Result: race conditions and lost work.

Fix: state clearly what is shared, add file ownership or worktrees where needed, and use locks for shared resources.

Permission fatigue

Symptom: repeated approval prompts for the same action.

Result: users click through without reading.

Fix: scoped approvals, standing modes chosen by the user, remembered denials, and clear targets.

Model-owned state

Symptom: all coordination exists only in prompts.

Result: restarts lose queues, meetings, and commitments.

Fix: persist operational state in host services.

ARCHITECTURE FIELD GUIDE

Part 13: What the video simplified

Video phrasing	Repository-grounded correction
"Each agent has its own computer."	Agents share one sandbox computer and can receive separate private desktop windows or monitors.
"After a few approvals it adds the action to accepted permissions."	Local-computer access has explicit Always , Ask , and Never modes. Allow once is scoped. Repetition alone does not create standing consent.
"A manager is assigned to the meeting."	A manager is a useful role design, but the inspected orchestrator itself is a bounded rotating round-robin with mentions and pass behavior.
"Import your skills from Codex or Claude Code."	The inspected workflow importer clearly handles Markdown, live sources, CLAUDE.md , AGENTS.md , .claude/CLAUDE.md , and Cursor rules. Full skill-folder import needs a separate adapter.
"The Cursor team accidentally leaked parts by mistake."	The evidence supports a public app artifact and an unofficial reconstruction. Cursor has not authenticated the repository as its original source or confirmed an accidental repository publication.
"Reverse engineered my own version."	The safer description is a separate implementation inspired by public behavior and architectural claims, using official provider SDKs and an original interface.

Corrections make the guide more useful. A builder needs the real boundary, not the shortest sentence from a video.

ARCHITECTURE FIELD GUIDE

Part 14: Glossary

Agent: a durable identity, transcript, memory view, tools, and run state wrapped around a model.

Agent lane: work triggered by another agent.

Automation lane: work triggered by a schedule or rule.

Background wake: a hidden turn caused by completed work, an inbound message, or another non-user event.

Computer-use window: a desktop display and control surface assigned to an agent on a shared sandbox computer.

Envelope: the stored representation of a message, including sender, target, text, priority, and time.

Epoch: a version marker used to stop stale work after the room or request has changed.

Fire-and-forget: delivery returns an acknowledgement immediately; the recipient's answer arrives later.

Group room: a shared transcript where multiple agents can contribute in bounded turns.

Hidden prompt: host-generated context that wakes an agent without pretending the user typed it.

Model adapter: a provider-specific bridge that exposes a common streaming and tool-call contract.

Priority message: a one-to-one peer message allowed to supersede non-user work.

Run scheduler: the service that serializes, orders, cancels, and resumes agent turns.

Sandbox: the controlled computer environment where agent tools run.

Scoped approval: permission tied to a specific action, target, agent, and run scope.

Transcript: the chronological conversation and tool record for an agent or room.

ARCHITECTURE FIELD GUIDE

Part 15: Final blueprint

If you remember only ten rules, remember these:

- 1 The model is only one layer.
- 2 Give every agent a stable ID and isolated transcript.
- 3 Keep profile, transcript, and memory separate.
- 4 Send small asynchronous handoffs instead of sharing every thought.
- 5 Treat a reply as a later event, not the return value of a message call.
- 6 Protect the user lane from internal interruption.
- 7 Bound meetings and let agents pass.
- 8 Distinguish a private screen from a private computer.
- 9 Make consent explicit, scoped, and auditable.
- 10 Persist coordination state outside the model prompt.

That is the system underneath the spectacle: a small company made from queues, identities, notebooks, policies, and computers, with language models working inside the structure.

For the exact repository paths and line ranges that support this guide, open [EVIDENCE-MAP.md](#). To adapt the design to an existing agent, use [prompts/EXTEND-YOUR-AGENT-SYSTEM.md](#) with this file attached.

Part 16: Repository source map

This is the shortest path from the guide back to the inspected implementation. Line numbers refer to commit [a9f633e09d49a85829b8236331b9e21f7e612634](#). Open [EVIDENCE-MAP.md](#) for the complete claim-by-claim ledger.

Mechanism	Primary repository anchor	What to verify
Provenance	README.md:5-24 , PROVENANCE.md:22-48	Reconstruction boundary and artifact basis.
Identity and sessions	source/host/agents/agent-profile.ts:4-47 ; source/host/extensions/session/agent-session.ts:93-195	Profile fields, directories, database, and transcript owner.
Memory	source/host/runner/sand-memory.ts:1-175, 284-337	Tiers, prompt budgets, retrieval, and scoped shards.
Messages and wakes	source/host/agents/agent-messaging.ts:35-117 ; source/host/extensions/transcript/agent-to-agent-messaging.ts:93-335	Asynchronous delivery, records, later wakes, and priority.
Group rooms	source/host/extensions/transcript/group-chat-orchestrator.ts:31-105	Bounds, rotating speakers, pass behavior, and caps.
Separate screens	source/host/box/shared-desktop-sand-box.ts:11-36	Shared computer, window map, accessors, and VNC URLs.
Permissions	source/host/extensions/local-tool-permission/local-tool-permission-controller.ts:20-86	Resolutions, expiry, and scope retirement.
Workflows and providers	source/host/workflows/workflow-store.ts:25-55 ; source/node-agent-coordinator/inference-router.ts:10-184	Import boundaries and reconstruction-author routing.

How to verify a claim without getting lost

- 1 Start with the classification in [EVIDENCE-MAP.md](#).
- 2 Open the exact pinned line range above.
- 3 Read the type or schema before reading orchestration logic so you know what the data can represent.
- 4 Follow writes into storage and then follow the later read or wake path.
- 5 Check whether the repository labels the feature as an author addition before attributing it upstream.

The source map is intentionally narrow. It points to the decisions that create the visible product behavior, not every supporting utility in the repository.